National Centre of Artificial Intelligence (NCAI), NUST, Islamabad

Certified Artificial Intelligence Professional (CAIP)

Final Project Report · Batch 8 · 24 August 2026

Predicting Maintenance Cost for WAPDA Staff Houses

A simple comparison of models on the WAPDA data model

Saeed Ahmad

Machine learning and data governance

Abstract

This project asks a simple question: for one WAPDA staff-colony house or apartment, what will building maintenance cost in the next period?

The dataset follows the **WAPDA data model** from the WASC residential inventory (101 houses and apartments in categories A–F). One row is one property on one date. The target is eligible maintenance cost for the next twelve months. Shared building costs are split equally across occupied units. "High cost" means the top 20% of costs in the training data.

WAPDA did not release linked work-order and invoice files for training. So we filled the same WAPDA data model with official American Housing Survey (AHS) public data from 2015–2023. We have 36,623 housing units and 86,125 before/after pairs. These are U.S. survey answers mapped to the WAPDA schema. They are not WAPDA bills, and they are not in Pakistani rupees.

We split the data so the same house never appears in both training and testing. We only learn cleaning rules and the high-cost cutoff (USD 1,428) from training data. We compare simple baselines with linear regression, random forest, gradient boosting, and XGBoost.

Main results. On the validation set, the type-median baseline has the lowest average error (MAE = 827.95 USD). On the test set, tuned XGBoost has the lowest average error among models that still find expensive houses (MAE = 944.34 USD; F1 = 0.391). Absolute-error XGBoost looks better on average error (MAE = 899.07 USD) but almost stops finding expensive houses (F1 = 0.113). These numbers are for the WAPDA-modelled public dataset. They are not a final WAPDA forecast.

Keywords: WAPDA staff colonies, maintenance cost, model comparison, XGBoost

1. Introduction

WAPDA looks after houses and apartments in staff colonies. Budgets often use past averages, inspections, and complaints. That can miss big differences between properties. An old house with many plumbing problems is not the same as a recently repaired apartment of the same type.

Our goal is clear:

> Given what we know about one property today, estimate its eligible building maintenance cost for the next twelve months.

Eligible work includes normal repair, corrective work, and emergency building work. It also includes a fair share of shared building work. We exclude major renovation, rebuilding, personal appliances, land value, and rent.

The WASC inventory has **101 residential units** in categories A–F (counts 1, 4, 16, 24, 16, and 40). That inventory defines the WAPDA data model. The repair ledger alone was not enough for machine learning, because most costs are not linked to one house.

So we built a training dataset **on the WAPDA data model**, then filled it with public housing survey data that can be mapped to the same fields. Section 2 explains the data and methods. Section 3 shows the results. Section 4 explains what the results mean.

This work covers two CAIP areas: machine learning, and careful data handling. A web app was planned; it is not built yet.

2. Methodology

2.1 What is the WAPDA data model?

Think of the WAPDA data model as the rulebook for one training row:

- **One row** = one house or apartment on one cutoff date.
- **What we predict** = eligible maintenance cost for the next 12 months.
- **Included** = civil, plumbing, electrical, roof, paint, sewer, and emergency building work.
- **Excluded** = major renovation, rebuilding, appliances, land, and rent.
- **Scope** = the 101 WASC residential units (not offices, roads, mosque, hostel, or hospital).
- **High-cost house** = cost in the top 20% of the training set.
- **Inputs** = only facts known on or before the cutoff date (type, age, size, condition, past maintenance).

WASC files stay private and only define this rulebook. They are not the big training table.

Because WAPDA work orders were not available, we used official AHS survey files for 2015, 2017, 2019, 2021, and 2023 [1]–[4]. For each house we take facts from one survey year and the maintenance cost from the next survey year (two years later). Example: 2019 facts → 2021 cost. We kept 36,623 houses and 86,125 such pairs (more than the 500-property minimum).

Important limit: this is a **WAPDA-shaped dataset**, not real WAPDA invoices. Costs are in U.S. dollars from a survey. In 2023 the survey raised its maximum reportable maintenance amount from USD 10,000 to USD 100,000 [4]. We do not convert dollars to PKR.

Table 1 shows how some WAPDA fields map to AHS fields. Past cost comes from the earlier survey. The target cost comes from the later survey.

Table 1: How WAPDA fields map to AHS survey fields.

WAPDA field	AHS field	Meaning
Building type	`BLD`	House type (used by type median)
Year built	`YRBUILT`	Age of the unit
Rooms / bedrooms	`TOTROOMS`, `BEDROOMS`	Size (no exact floor area)
Past maintenance cost	earlier `MAINTAMT`	Known at cutoff
Future cost (target)	later `MAINTAMT`	What we predict
Condition codes	roof/sewer flags	Repair problems

2.2 How we split and clean the data

We put each house into only one group: training, validation, or test (Table 2). The same house cannot teach the model and also be used to grade it. The main test includes 2023 costs. A second "sensitivity" test removes 2023, so a survey-rule change is not confused with model skill.

Cleaning (filling missing values, encoding categories) uses training data only. The first feature matrix has 205 columns. Later we added five simple derived features (age, log past cost, cost per room, rooms per bedroom, defect count) and reached 215 columns. The high-cost cutoff is USD 1,428 from training costs only.

Table 2: Data split by house (not by row).

Set	Houses	Rows	Used for
Training	10,103	13,871	Learn cleaning rules and models
Validation	7,067	15,400	Choose settings (lowest MAE)
Test	19,453	56,854	Final check
Total	36,623	86,125	

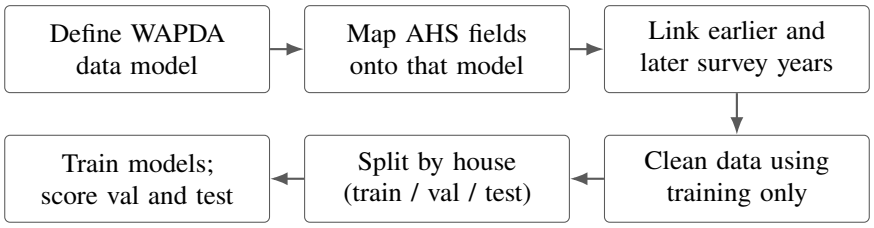

Figure 1: Project steps. First we fix the WAPDA data model. Then we map public survey data onto it.

2.3 Models and how we choose a winner

We compare three simple baselines and four learning models:

- **Training median:** predict the middle cost from training.
- **Type median:** predict the middle cost for that building type.
- **Prior cost:** predict last survey's maintenance cost.
- **Learning models:** linear regression, random forest, gradient boosting [5]–[7], and XGBoost [8].

First we used fixed settings. Later we tuned tree settings on the validation set only (144 XGBoost trials; best: 200 trees, depth 4, learning rate 0.03). We also tried absolute-error training. We never tuned on the test set.

Metrics in plain words:

- **MAE** = average dollar miss (main score). Lower is better.
- **RMSE** = similar, but big misses hurt more.
- **High-cost F1** = how well we find houses above USD 1,428. Higher is better.

Our written rule: pick by validation MAE. Also report the no-2023 test and high-cost F1. A learning model is "promoted" only if it wins validation MAE and is at least as good on the no-2023 test. A good test MAE alone is not enough.

3. Implementation and results

The code is a Python package named `caip_maintenance`. Config files sit under `configs/`. Tests check that houses do not leak across splits and that cleaning uses training data only. The tables below use the saved experiment scores. We do not print private or row-level survey data here.

3.1 Main comparison

Table 3 and Figures 2–4 show the key systems. "Original" = first feature set. "Tuned" = engineered features + validation search.

On validation, **type median** wins (827.95 USD). Tuned XGBoost is close (838.83 USD) but still about 11 dollars worse.

On the test set, **tuned XGBoost** wins among normal models (944.34 USD MAE, 2182.07 USD RMSE).

Table 3: Main scores in USD (except F1). Lower MAE/RMSE is better. Higher F1 is better. Median baselines have no F1 because they never flag high-cost houses.

System	Val. MAE	Test MAE	No-2023 MAE	Test RMSE	Test F1
Training median	832.16	965.85	828.69	2360.09	—
Type median	**827.95**	962.81	**824.82**	2361.65	—
Prior cost	952.44	1058.25	936.53	2358.81	**0.410**
Linear regression (orig.)	849.50	950.91	833.34	2182.90	0.384
Random forest (orig.)	850.27	956.54	837.78	2188.13	0.397
Gradient boosting (orig.)	843.34	949.15	829.68	2188.68	0.356
XGBoost (orig.)	845.90	949.38	830.26	2189.87	0.386
Random forest (tuned)	841.23	946.57	828.18	2186.38	0.385
Gradient boosting (tuned)	841.16	945.27	826.76	2183.34	0.373
XGBoost (tuned)	838.83	**944.34**	825.35	**2182.07**	0.391
XGBoost (absolute loss)	778.41	899.07	767.77	2268.28	0.113

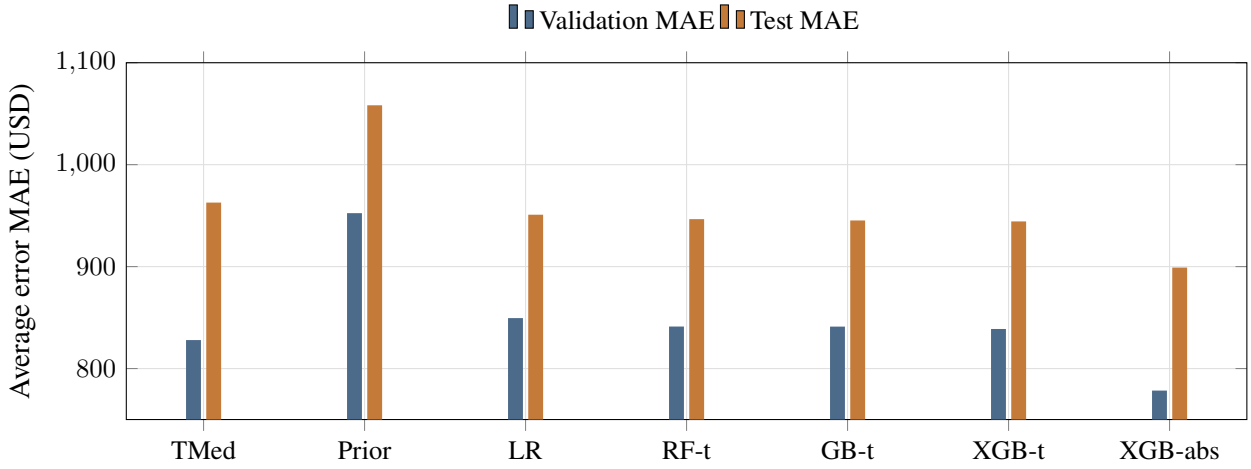

Figure 2: Average error (MAE). Type median (TMed) is best on validation. Tuned XGBoost (XGB-t) is best on the test set among normal models. Absolute-error XGBoost (XGB-abs) has the lowest MAE but fails later on high-cost finding.

That is about 18 dollars better than type median. Random forest and gradient boosting are close behind. Adding extra features did not help linear regression.

All methods look better when we remove 2023 rows. That mostly reflects the 2023 survey change, not a big model skill gap. Type median is still slightly best on that no-2023 test. So our written rule does **not** promote XGBoost as the official winner, even though it wins the full test set.

Learning models also cut RMSE a lot (about 2,360 down to about 2,182). That means they handle some very expensive cases better than a simple median, even when average error only improves a little.

3.2 Finding expensive houses

Median baselines never predict above USD 1,428, so they never mark a house as high cost.

Among scoring methods, **prior cost** is best at finding expensive houses (F1 = 0.410). Tuned XGBoost is close (F1 = 0.391). Absolute-error XGBoost looks good on MAE but is bad at this job (F1 = 0.113). It rarely predicts high values, so it misses most expensive houses (Table 4, Figures 3–4).

Two quick checks: training on $\log(1 + \text{cost})$ helped middle errors but hurt high-cost finding, so we did not keep it. Removing past-cost features made gradient boosting worse (test MAE from 947.79 to 979.67). Past maintenance cost matters.

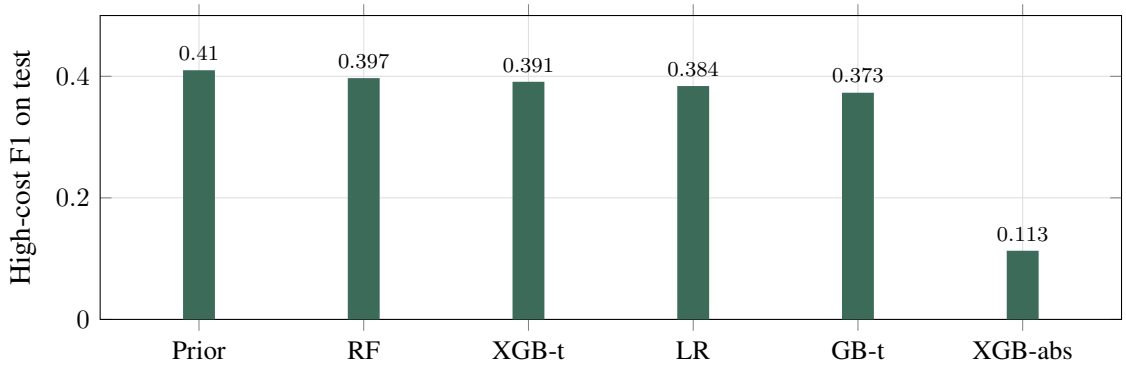

Figure 3: How well each method finds expensive houses. Prior cost is best. Absolute-error XGBoost is much worse.

Table 4: Finding houses above USD 1,428 on the test set.

System	Precision	Recall	F1
Prior cost	0.456	0.373	0.410
Random forest (orig.)	0.476	0.340	0.397
XGBoost (tuned)	0.502	0.320	0.391
Linear regression (orig.)	0.508	0.309	0.384
Gradient boosting (tuned)	0.515	0.293	0.373
XGBoost (absolute loss)	0.634	0.062	0.113

3.3 What should we report?

By our written rule:

- **Best simple cost estimate:** type median.
- **Best simple high-cost signal:** prior cost.

If we only ask which learning model is strongest on the full test set *and* still finds expensive houses, the answer is **tuned XGBoost**. That is a useful finding, not a claim that WAPDA should deploy it tomorrow.

4. Discussion and conclusion

A simple type median is hard to beat when the split is fair. House type already carries a lot of information. Learning models improve the test score a little and give a usable high-cost score. Absolute error reduces average miss but mostly stops warning about expensive houses. That is the wrong trade for prioritisation.

We need two different answers for two different jobs:

1. **Budget number:** type median is strongest under the validation rule.
2. **Which houses look expensive:** prior cost is the clearest simple signal; tuned XGBoost is the best learning compromise.

The 2023 survey change shows why a second test without 2023 was needed. If recording rules change, every error number can move even when houses stay the same.

Privacy also shaped the work. Names, CNIC, phone numbers, salaries, and free-text remarks from WASC files are not in the training tables or this report. IDs are tokenized. Raw rows and trained model files are not in the email annex.

For real WAPDA use later, the useful part is the data model and the safe workflow, not these USD scores. WAPDA would still need linked work orders, verified costs, and clear coverage so “zero cost” is not confused with “missing data.”

The web app is future work.

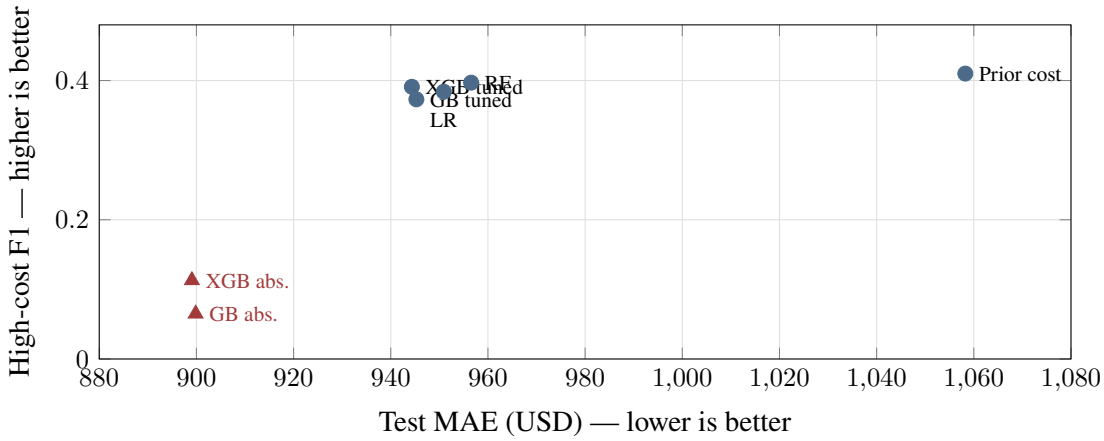

Figure 4: Average error vs finding expensive houses. Absolute-error models move left (better MAE) but down (worse F1).

Final takeaway. The dataset is made on the WAPDA data model and filled with mapped AHS records because WAPDA work-order files were not available. Type median is the main cost baseline. Prior cost is the main high-cost baseline. Tuned XGBoost is the best learning model on the full test set that still finds expensive houses. Absolute-error models lower MAE but miss most expensive houses. None of these is a validated WAPDA operational forecast.

References

[1] U.S. Census Bureau and U.S. Department of Housing and Urban Development, "American Housing Survey: 2023 Data," 2023. [Online]. Available: `https://www.census.gov/programs-surveys/ahs/data/2023.html`

[2] U.S. Census Bureau, "American Housing Survey Codebooks." [Online]. Available: `https://www.census.gov/programs-surveys/ahs/tech-documentation/codebooks.html`

[3] U.S. Census Bureau, *Sample Case History File 2015 to 2023*. [Online]. Available: `https://www2.census.gov/programs-surveys/ahs/documentation/`

[4] U.S. Census Bureau, *2023 AHS Historical Changes*, 2023. [Online]. Available: `https://www2.census.gov/programs-surveys/ahs/2023/2023%20AHS%20Historical%20Changes.pdf`

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[7] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.

A. Code annex (summary)

Full code is in `caip_code_annex.zip`. The separate test file is `caip_final_source.py`. Neither file has raw WASC or AHS row data.

Example checks from the project root:

```
PYTHONPATH=src .venv/bin/python -m caip_maintenance.data \
  audit-experiment --experiment ahs-baselines-models-v1
```

```
PYTHONPATH=src .venv/bin/python -m caip_maintenance.data \
  audit-ahs-xgboost-tuning --experiment ahs-xgboost-tuning-v1
.venv/bin/python -m unittest discover -s tests -v
```